

# Testing Architecture for TLS Fingerprinting

<https://gemini.google.com/app/3084077766c8f598>

```
User prompt: #include "tlsfp/capture.hpp" #include "tlsfp/parser.hpp" #include <iostream> #include <netinet/ip.h>
#include <netinet/ip6.h> #include <netinet/tcp.h> #include <netinet/in.h> #include <arpa/inet.h> namespace tlsfp {
static pcap_t *g_pcap_handle = nullptr; void signal_handler(int signum) { if (g_pcap_handle != nullptr) { std::cout <<
"\n[*] Caught signal " << signum << ", stopping capture gracefully...\n"; pcap_breakloop(g_pcap_handle); } } void
packet_callback(u_char *user_data, const struct pcap_pkthdr *pkthdr, const u_char *packet) { CaptureContext *ctx =
reinterpret_cast<CaptureContext*>(user_data); if (!ctx) return; // 1. Resolve Link-Layer Offset dynamically size_t
link_header_len = 0; switch (ctx->link_type) { case DLT_EN10MB: // Standard Ethernet / Wi-Fi (14 bytes)
link_header_len = 14; break; case DLT_LINUX_SLL2: // Linux Cooked v2 (WSL / tcpdump -i any) (20 bytes)
link_header_len = 20; break; case DLT_LINUX_SLL: // Linux Cooked v1 (16 bytes) link_header_len = 16; break; case
DLT_NULL: // BSD / macOS Loopback (4 bytes) link_header_len = 4; break; case DLT_RAW: // Raw IP (VPN / tun
interfaces) (0 bytes) link_header_len = 0; break; default: return; // Unsupported link-layer } // Ensure packet has at least
1 byte after link layer to check the IP version nibble if (pkthdr->caplen < link_header_len + 1) { return; } // 2. Determine
IP Version (first 4 bits of the IP header) uint8_t ip_version = (packet[link_header_len] >> 4); size_t ip_header_len = 0;
char src_ip[INET6_ADDRSTRLEN]; char dst_ip[INET6_ADDRSTRLEN]; FlowKey key{}; if (ip_version == 4) { // --- IPv4
Path --- if (pkthdr->caplen < link_header_len + sizeof(struct ip)) return; const struct ip *ip_hdr = reinterpret_cast<const
struct ip *>(packet + link_header_len); if (ip_hdr->ip_p != IPPROTO_TCP) return; // Verify TCP protocol ip_header_len
= ip_hdr->ip_hl * 4; if (ip_header_len < 20) return; inet_ntop(AF_INET, &(ip_hdr->ip_src), src_ip,
INET_ADDRSTRLEN); inet_ntop(AF_INET, &(ip_hdr->ip_dst), dst_ip, INET_ADDRSTRLEN); // Store binary IP
addresses directly (Zero Heap Allocation) key.ip_version = 4; key.src_ip.v4 = ip_hdr->ip_src; key.dst_ip.v4 = ip_hdr-
>ip_dst; } else if (ip_version == 6) { // --- IPv6 Path --- if (pkthdr->caplen < link_header_len + sizeof(struct ip6_hdr))
return; const struct ip6_hdr *ip6_hdr = reinterpret_cast<const struct ip6_hdr *>(packet + link_header_len); if (ip6_hdr-
>ip6_nxt != IPPROTO_TCP) return; // Verify TCP protocol ip_header_len = sizeof(struct ip6_hdr); // Fixed 40 bytes for
standard IPv6 inet_ntop(AF_INET6, &(ip6_hdr->ip6_src), src_ip, INET6_ADDRSTRLEN); inet_ntop(AF_INET6, &
(ip6_hdr->ip6_dst), dst_ip, INET6_ADDRSTRLEN); // Store binary IP addresses directly (Zero Heap Allocation)
key.ip_version = 6; key.src_ip.v6 = ip6_hdr->ip6_src; key.dst_ip.v6 = ip6_hdr->ip6_dst; } else { return; // Non-IP packet }
// 3. Inspect TCP Header if (pkthdr->caplen < link_header_len + ip_header_len + sizeof(struct tcphdr)) { return; } const
struct tcphdr *tcp_hdr = reinterpret_cast<const struct tcphdr *>(packet + link_header_len + ip_header_len); size_t
tcp_header_len = tcp_hdr->th_off * 4; if (tcp_header_len < 20) { return; } // Set 5-tuple ports key.src_port = tcp_hdr-
>th_sport; key.dst_port = tcp_hdr->th_dport; // 4. Calculate Payload Offset & Length size_t header_total_len =
link_header_len + ip_header_len + tcp_header_len; if (pkthdr->caplen <= header_total_len) { return; // Pure TCP
ACK/SYN/FIN or header-only frame } size_t payload_len = pkthdr->caplen - header_total_len; const uint8_t *payload =
packet + header_total_len; // 5. TCP Sequence Alignment and Deduplication StreamBuffer &buf = ctx-
>active_flows[key]; buf.last_seen = pkthdr->ts.tv_sec; uint32_t seq = ntohl(tcp_hdr->th_seq); if (!buf.seq_initialized) {
buf.next_seq = seq; buf.seq_initialized = true; } if (seq < buf.next_seq) { // Handle duplicate bytes / retransmissions
uint32_t diff = buf.next_seq - seq; if (diff >= payload_len) return; // Entirely duplicate, discard payload += diff;
payload_len -= diff; } else if (seq > buf.next_seq) { // Gap detected due to out-of-order packet: clear flow state ctx-
>active_flows.erase(key); return; } if (buf.len + payload_len > StreamBuffer::MAX_BUF_SIZE) { ctx-
>active_flows.erase(key); return; } std::memcpy(buf.bytes + buf.len, payload, payload_len); buf.len +=
static_cast<uint16_t>(payload_len); buf.next_seq += payload_len; // Advance expected sequence number // 6. Inspect
TLS Record Header if (buf.len < 5) { return; } // Wait for full 5-byte TLS record header } uint8_t record_type =
buf.bytes[0]; uint8_t major_ver = buf.bytes[1]; uint8_t minor_ver = buf.bytes[2]; // Declared here so it remains in scope
for validation below uint16_t tls_rec_len = (static_cast<uint16_t>(buf.bytes[3]) << 8) | buf.bytes[4]; bool is_valid_tls =
(record_type == 0x16) && (major_ver == 0x03) && (minor_ver <= 0x04) && (tls_rec_len <= 16384); if (!is_valid_tls) {
ctx->active_flows.erase(key); return; } size_t total_expected_len = 5 + tls_rec_len; if (buf.len < total_expected_len) {
return; // Wait for full TLS record payload } if (tls_rec_len < 4 || buf.len < 9) { ctx->active_flows.erase(key); return; } //
Process Complete Reassembled ClientHello (0x01) or ServerHello (0x02) uint8_t handshake_type = buf.bytes[5]; if
(handshake_type == 0x01 || handshake_type == 0x02) { uint32_t hs_len = (static_cast<uint32_t>(buf.bytes[6]) << 16) |
(static_cast<uint32_t>(buf.bytes[7]) << 8) | static_cast<uint32_t>(buf.bytes[8]); // Validate handshake length against
tls_rec_len if (4 + hs_len > tls_rec_len) { ctx->active_flows.erase(key); return; } if (ctx->dumper != nullptr) {
pcap_dump(reinterpret_cast<u_char*>(ctx->dumper), pkthdr, packet); pcap_dump_flush(ctx->dumper); } uint16_t
src_port = ntohs(tcp_hdr->th_sport); uint16_t dst_port = ntohs(tcp_hdr->th_dport); std::cout << "[+] Captured " <<
(handshake_type == 0x01 ? "ClientHello" : "ServerHello") << " | Flow: [" << src_ip << ":" << src_port << " -> [" << dst_ip
<< ":" << dst_port << " | Reassembled Size: " << buf.len << " bytes" << " | Handshake Payload: " << hs_len << "
bytes\n"; ctx->active_flows.erase(key); } else { ctx->active_flows.erase(key); } } bool start_capture(const
CaptureOptions &opts) { char errbuf[PCAP_ERRBUF_SIZE]; pcap_t *handle = nullptr; if (!opts.read_filename.empty()) {
handle = pcap_open_offline(opts.read_filename.c_str(), errbuf); } else { handle =
pcap_open_live(opts.interface_name.c_str(), BUFSIZ, 1, 1000, errbuf); } if (!handle) { std::cerr << "[+] Error opening
pcap target: " << errbuf << "\n"; return false; } g_pcap_handle = handle; struct bpf_program fp; if (pcap_compile(handle,
&fp, "tcp port 443", 0, PCAP_NETMASK_UNKNOWN) == 0) { pcap_setfilter(handle, &fp); pcap_freecode(&fp); } else {
std::cerr << "[+] Warning: Failed to set BPF filter 'tcp port 443'\n"; } CaptureContext ctx; ctx.link_type =
pcap_datalink(handle); std::cout << "[*] Capture initialized. Datalink type: " << ctx.link_type << "\n"; if
(opts.write_filename.empty()) { ctx.dumper = pcap_dump_open(handle, opts.write_filename.c_str()); if (!ctx.dumper) {
std::cerr << "[+] Warning: Could not open output PCAP for writing: " << opts.write_filename << "\n"; } }
pcap_loop(handle, 0, packet_callback, reinterpret_cast<u_char*>(&ctx)); if (ctx.dumper) {
pcap_dump_flush(ctx.dumper); pcap_dump_close(ctx.dumper); } pcap_close(handle); g_pcap_handle = nullptr;
```

```
std::cout << "[*] Capture completed.\n"; return true; } } // namespace tlsfp #include "../include/tlsfp/capture.hpp" #include
<iostream> #include <unistd.h> #include <csignal> #include <filesystem> namespace fs = std::filesystem; static void
print_usage(const char *prog_name) { std::cerr << "Usage: " << prog_name << " [OPTIONS]\n" << "Options:\n" << "-i
<interface> Live network interface (eg, eth0, any)\n" << "-r <pcap_file> Read packets from offline PCAP file\n" << "-w
<pcap_file> Save matched TLS handshakes to output PCAP\n" << "-h Show this help message\n"; } int main(int argc,
char *argv[]) { if (argc < 2) { print_usage(argv[0]); return 1; } tlsfp::CaptureOptions opts; int opt; while ((opt =
getopt(argc, argv, "i:r:w:h")) != -1) { switch (opt) { case 'i': opts.interface_name = optarg; break; case 'r':
opts.read_filename = optarg; break; case 'w': opts.write_filename = optarg; break; case 'h': print_usage(argv[0]); return
0; default: print_usage(argv[0]); return 1; } } if (!opts.write_filename.empty()) { fs::path p(opts.write_filename); if
(!p.has_parent_path()) { // Locate ../././pcaps relative to build directory fs::path target_dir = ".././pcaps"; if
(!fs::exists(target_dir)) { fs::create_directories(target_dir); } opts.write_filename = (target_dir / p).string(); } } if
(opts.interface_name.empty() && opts.read_filename.empty()) { std::cerr << "[*] Error: Must specify an interface (-i) or
PCAP file (-r).\n"; print_usage(argv[0]); return 1; } std::signal(SIGINT, tlsfp::signal_handler); std::signal(SIGTERM,
tlsfp::signal_handler); if (!tlsfp::start_capture(opts)) { std::cerr << "[*] Capture engine encountered a fatal error.\n"; return
1; } return 0; } I also have ja3 and ja3s which I need to test, so that is also important, I will need to do the best
mechanisms to test the code. Like there can be multiple tests right? I need to create unit tests for each one of them,
maybe 1 testing the ja3 logic, 1 testing the ingestion of different agents, one testing client hello and server hello which
is kept in multiple packets and a lot of edge cases, tell me what can I do to do this. I will give you more context if
needed. I already have a cmake file and maybe we can create ctest here so that I can I can be sure that each and
every part of my project works independently and also can work as a pipeline
```

Response: Structuring your test suite into decoupled unit tests for core parsing/fingerprinting logic and CTest-driven PCAP integration tests is the standard approach for network protocol analysis projects.

### Testing Architecture Overview

- **Unit Tests (Pure Logic):** Test binary parsing functions and JA3/JA3S string generation directly with static byte arrays. Do not involve network interfaces or pcap files at this layer.
- **State Machine / Stream Tests:** Feed synthetic TCP packets (fragmented, out-of-order, or retransmitted) into `packet_callback` or `StreamBuffer` to test TCP reassembly robustness.
- **PCAP Integration Tests (CTest):** Execute your compiled binary against offline PCAP fixtures (`-r sample.pcap`) generated from real user-agents (Chrome, Firefox, Curl, Python) and verify stdout or dumped PCAPs.

### Essential Test Cases Matrix

| Category                   | Test Scenario                         | Target Verification                                                                                                                         |
|----------------------------|---------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <b>JA3 Fingerprinting</b>  | Filtering GREASE values               | Verify values like <code>0x0A0A</code> , <code>0x1A1A</code> , etc., are stripped from ciphers, extensions, and curves before MD5 hashing.  |
| <b>JA3 Fingerprinting</b>  | Extension & Curve ordering            | Ensure original byte sequence of extensions and elliptic curves is strictly preserved.                                                      |
| <b>JA3S Fingerprinting</b> | ServerHello Parsing                   | Verify extraction of TLS Version, Cipher Suite, and Extensions list from ServerHello (type <code>0x02</code> ).                             |
| <b>TCP Reassembly</b>      | Multi-packet ClientHello              | Send ClientHello fragmented across 2–3 TCP packets; verify buffer waits until full record length is assembled.                              |
| <b>TCP Reassembly</b>      | Duplicate / Retransmitted payload     | Send overlapping <code>th_seq</code> numbers ( <code>seq &lt; next_seq</code> ); verify duplicate bytes are sliced without corrupting data. |
| <b>TCP Reassembly</b>      | Out-of-order delivery / Sequence gaps | Send <code>seq &gt; next_seq</code> ; ensure session memory is reset and no heap buffer overflow occurs.                                    |
| <b>Link-Layer &amp; IP</b> | Packet Headers (SLL2, IPv6)           | Test <code>DLT_LINUX_SLL2</code> , <code>DLT_EN10MB</code> , and IPv6 extension headers to confirm dynamic offset calculation works.        |
| <b>Malformed Packets</b>   | Payload Length Mismatch               | Test <code>hs_len &gt; tls_rec_len</code> or <code>tls_rec_len &gt; 16384</code> to ensure memory is safely reclaimed without crashing.     |

### Decoupling Parsing Logic for Testing

To make JA3 and TLS parsing independently unit-testable without relying on libpcap types (`pcap_pkthdr`), split parsing into standalone pure functions:

```
// tlsfp/parser.hpp
struct TLSClientHello {
    uint16_t client_version;
    std::vector<uint16_t> cipher_suites;
    std::vector<uint16_t> extensions;
    std::vector<uint16_t> elliptic_curves;
    std::vector<uint8_t> ec_point_formats;
};

// Pure function: operates directly on raw bytes
bool parse_client_hello(const uint8_t *data, size_t len, TLSClientHello &out);

// Fingerprint calculation functions
std::string generate_ja3_string(const TLSClientHello &ch);
std::string compute_md5(const std::string &input);
```

## CTest & CMake Setup

Integrate unit testing and offline PCAP integration testing in your `CMakeLists.txt`:

```
cmake_minimum_required(VERSION 3.14)
project(tlsfp CXX)

set(CMAKE_CXX_STANDARD 17)

# Enable CTest framework
enable_testing()

# Core library source files
add_library(tlsfp_core STATIC
    src/parser.cpp
    src/capture.cpp
)
target_include_directories(tlsfp_core PUBLIC include)
find_package(OpenSSL REQUIRED)
target_link_libraries(tlsfp_core PUBLIC OpenSSL::Crypto pcap)

# Main Binary
add_executable(tlsfp_bin src/main.cpp)
target_link_libraries(tlsfp_bin PRIVATE tlsfp_core)

# --- UNIT TESTS (e.g., using GoogleTest or Catch2) ---
find_package(GTest REQUIRED)

add_executable(unit_tests
    tests/test_ja3.cpp
    tests/test_reassembly.cpp
)
target_link_libraries(unit_tests PRIVATE tlsfp_core GTest::gtest_main)

# Register CTest target for unit tests
add_test(NAME RunUnitTests COMMAND unit_tests)

# --- INTEGRATION TESTS (Using pre-recorded PCAP fixtures) ---
# Verify Chrome JA3 output
add_test(
    NAME Integration_Chrome_JA3
    COMMAND tlsfp_bin -r ${CMAKE_SOURCE_DIR}/tests/fixtures/chrome_client_hello.pcap
)
set_tests_properties(Integration_Chrome_JA3 PROPERTIES
    PASS_REGULAR_EXPRESSION "Captured ClientHello.*Reassembled Size:"
)

# Verify Multi-packet reassembly fixture
add_test(
    NAME Integration_Fragmented_TLS
    COMMAND tlsfp_bin -r ${CMAKE_SOURCE_DIR}/tests/fixtures/fragmented_client_hello.pcap
)
set_tests_properties(Integration_Fragmented_TLS PROPERTIES
    PASS_REGULAR_EXPRESSION "Captured ClientHello"
```

)

### Example Unit Test File (tests/test\_ja3.cpp)

```
#include <gtest/gtest.h>
#include "tlsfp/parser.hpp"

TEST(JA3Test, StripsGreaseValues) {
    // ClientHello with GREASE cipher suite 0x0A0A and TLS_AES_128_GCM_SHA256 (0x1301)
    tlsfp::TLSCClientHello ch;
    ch.client_version = 0x0303;
    ch.cipher_suites = {0x0A0A, 0x1301, 0x1302};
    ch.extensions = {0x0000, 0x1A1A, 0x000D};
    ch.elliptic_curves = {0x001D};
    ch.ec_point_formats = {0x00};

    std::string ja3_str = tlsfp::generate_ja3_string(ch);

    // Expect 0x0A0A and 0x1A1A to be filtered out from raw string output
    EXPECT_EQ(ja3_str, "771,4865-4866,0-13,29,0");
}
```

---